Member 1 – Text Preprocessing & Sectioning (aiml_core/text_pipeline.py)

What their code does (ML/NLP)

- normalize_text(text) turns raw JD / CV text into cleaned tokens (lowercased, regex-filtered).
- split_sections(text) splits resumes/JDs into sections like experience, education, skills based on headings.

Classic outputs they can show

- **Tokenization examples**:
  - Input: a messy JD snippet.
  - Output: normalized token string: "senior data engineer python sql spark aws".
- **Section split table**:
  - For a real resume, show:

Section	First 2–3 lines (preview)
experience	“Software Engineer – 3 years – Python, Django”
education	“BTech CSE, XYZ University, 2022”
skills	“Python, SQL, Docker, Kubernetes”

- They can also report **simple stats**:
  - Avg number of tokens per resume / per JD.
  - How many resumes had a skills section detected.

This is clearly NLP: text in, structured/clean text out.

Member 2 – Skills & Experience Extraction 2.0 (aiml_core/profile_extractor.py)

What their code does (ML/NLP)

- extract_skills_from_text(text) parses JD or resume text (especially the “Skills” section from Member 1) into a **deduplicated skill list**.

- estimate_experience_years(text) scans the resume for patterns like “3 years”, “2–3 years”, “5+ years” and returns a numeric **experience estimate**.

Classic outputs they can show

- **Skills extraction examples**:

Input line(s)	Extracted skills list
“Skills: Python, SQL; Docker, AWS”	["python", "sql", "docker", "aws"]
“Tech stack: React / Node / MongoDB”	["react / node / mongodb"] (per line/phrase)

- **Experience estimate table**:

Resume text snippet	Detected patterns	Estimated years
“3+ years of experience in Python…”	“3+ years”	3.0
“2–3 years in backend development”	“2-3 years”	3.0
“Worked since 2018 as a developer” (none)	none	None

- They can also summarize:
    - Histogram of estimated years for all candidates (e.g. how many 0–2, 3–5, >5).

This is obvious NLP: extracting structured skills and experience from free-text resumes/JDs.

Member 3 – TF-IDF Similarity & IR Model (aiml_core/similarity_models.py)

What their code does (ML/NLP)

- compute_tfidf_similarity(job_text, resumes):
    - Builds a TF-IDF representation of one job + multiple resumes.
    - Computes **cosine similarity** scores for each resume vs the job.

Classic outputs they can show

- **Similarity scores table** for one job:

Candidate	TF-IDF similarity
C1	0.82
C2	0.65
C3	0.30

- **Ranking list** just by similarity (before the more complex model):
    - "For job J123, our TF-IDF model ranks candidates: C1 > C2 > C3 based on cosine similarity."
- They can also:
    - Plot a **distribution** of similarity scores across all candidates for a job.
    - Show how similarity changes if they remove job title vs full JD.

This is standard IR/ML: vectorization + cosine similarity.

Member 4 – Learned Scoring Model (aiml_core/scoring_model.py)

What their code does (ML/NLP)

- Uses simple features per candidate:
    - sim = TF-IDF similarity (from Member 3).
    - coverage = skill coverage (from ranking/analytics).
- train_scoring_model(features, labels, out_path):
    - Trains a **LogisticRegression** model to predict 1 = good candidate / selected vs 0 = not selected.
- maybe_score_with_learned_model(sim, coverage, fallback_score, model_path):
    - When the model file exists, it returns P(good | sim, coverage) as the new ai_score.

Classic outputs they can show

- **Model weights**:

Feature	Coefficient (example)
sim	2.1
coverage	1.5

- (shows which feature matters more in the learned scoring).
- **Probability scores**:

Candidate	sim	coverage	Learned ai_score (prob)
C1	0.80	0.90	0.93
C2	0.60	0.70	0.74
C3	0.40	0.30	0.31

- They can also print a **ROC/PR curve** using this model if they want, but even just showing probabilities and weights is enough.

This is pure supervised ML: model training + inference.

Member 5 – Evaluation & Thresholding (aiml_core/evaluation.py + interview gating)

What their code does (ML/NLP)

- compute_binary_metrics(scores, labels, threshold):
    - Uses sklearn.metrics to calculate **precision, recall, F1, confusion matrix** for a given cut-off on ai_score.
- find_best_threshold(scores, labels):
    - Grid-searches thresholds and picks the one with **best F1**.
- Interview gating (already in interviews/views.py):
    - Uses predicted_label to enforce "Only AI-RECOMMENDED candidates can be scheduled".

Classic outputs they can show

- **Metric table** for threshold tuning:

Threshold	Precision	Recall	F1
0.4	0.70	0.85	0.77
0.5	0.78	0.80	0.79
0.6	0.85	0.65	0.74

  - And "Best F1 is at threshold 0.5".

- **Confusion matrix**:

[[TN, FP],

[FN, TP]]

e.g. [[120, 10],

[20, 50]]

- **Policy explanation**:
  - "We chose threshold 0.5; labels from Member 4's model above that become RECOMMENDED, and the interview API enforces that only RECOMMENDED are allowed into interviews."

This is **model evaluation & calibration**, a core ML task.

Member 6 – Match Analytics & Multi-Component Score (analytics.py)

What their code does (ML/NLP)

- compute_job_resume_match(job, resume):
  - Uses normalized text and extracted skills to compute:
    - skills_match_pct
    - experience_match_pct (using experience ratio)
    - education_match_pct (using education levels)
    - A weighted **overall_match_pct**.
  - Also returns required_skills, matched_skills, missing_skills.

Classic outputs they can show

- **Match breakdown per candidate** (like your screenshot):

Metric	Value
Overall match	72.40 %
Skills match	80.00 %
Experience match	60.00 %
Education match	90.00 %
Required skills	[python, sql, docker]
Matched skills	[python, sql]
Missing skills	[docker]

- Explanation:
    - "High education & skills, slightly low experience, which is why overall isn't 90+."

This is a **hand-crafted scoring model** built on NLP-extracted features.